Lecturers
Prof. Dr. **Florina M. Ciorba**
Prof. Dr. Heiko Schuldt
Prof. Dr. Christian Tschudin
Prof. Dr. Isabel Wagner

Assistants
Ali Ajorian
Thomas Jakobsche
Valentyna Pavliv
Yiming Wu

Info
https://adam.unibas.ch/go/crs/2206930
Pharmazentrum, U1075
Klingelbergstrasse 50, Basel

Exercise 3: OpenMP and MPI

Released: November 3, 2026
Exercise session: November 10, 2026
EvaExam: November 17, 2026
Contact: thomas.jakobsche@unibas.ch

Instructions

- This sheet is **not submitted and not graded**. Your exercise points come only from the EvaExam on November 17.
- The EvaExam consists of 10 KPrim questions (2 points each) about **this sheet**: what you observe when you run the code, and why. If you have worked through this sheet, you will be well prepared.
- You may discuss in groups, but run and modify the code yourself.
- All programs and job scripts are on ADAM in `exercise-processing.zip`. They are also printed at the end of this sheet. You only have to change a few lines yourself.

Timeline

- The **exercise session on November 10** is a hands-on introduction to OpenMP and MPI on miniHPC. Bring your laptop and make sure that the UniBas VPN works.
- **Set up your SSH key for miniHPC right away** (see below). It can take until the next day before you can log in.

Objectives

- Apply the four parallelization steps (decomposition, dependence analysis, mapping, parallel programming) to one example: computing π.
- Parallelize it for shared memory with OpenMP[1] (threads) and for distributed memory with MPI[2] (processes and messages), and compare both.
- Experience typical pitfalls: race conditions in shared memory, deadlocks in message passing.

[1] https://www.openmp.org
[2] MPI standard: https://www.mpi-forum.org; Open MPI, the MPI implementation on miniHPC: https://docs.open-mpi.org

Where to run your code

- Run everything (OpenMP and MPI) on miniHPC, so that you can compare your OpenMP and MPI measurements.

How to use miniHPC

- miniHPC is a computing cluster for research and teaching purposes (see https://hpc.dmi.unibas.ch/research/minihpc/).
- **SSH key (one-time setup).** Login with a password is disabled; miniHPC only accepts SSH keys. To register your public key:
  1. While connected to the UniBas VPN, log in to https://alis2.its.unibas.ch.
  2. Click the person icon (top right, next to the cog wheel) and select the tab *SSH Keys*.
  3. Click *Add a new SSH Key*, paste your **public** key, and enter a name that identifies this key for you. Leave *System Regex* empty (unless you do not want the key on all managed servers you have access to). Click *Add*.

  If you do not have a key yet, generate one with `ssh-keygen -t ed25519` (or follow the instructions on the ALIS site). Keys are stored in the hidden directory `.ssh` in your home directory, on all platforms; the public key is `id_ed25519.pub`. The key is transferred to the cluster overnight, possibly sooner. You can add several keys if you work from several computers. **Never share your private key** (the file without `.pub`).
- Login: `ssh <shortname>@cl-login.dmi.unibas.ch` (UniBas short name; you need to be connected to the UniBas VPN). To copy files, use `scp` (e.g., with `-r` for directories).
- The login node is shared by all users: you may edit and compile there, but **never execute your programs on the login node**. Submit them as jobs to the Slurm[3] resource manager instead, which allocates compute nodes to your job.
- Submit a job script with `sbatch <job_script>`, check its status with `squeue`, and cancel it with `scancel <job_id>`. The output is written to `slurm-<job_id>.out`. There is one job script per part of this sheet.
- The job scripts for time measurements (`job_omp.sh` and `job_pi_mpi.sh`) use `--exclusive`: your job gets whole nodes for itself, so other jobs do not disturb your time measurements. The price is that your job may have to **wait in the queue** until enough nodes are free. Submit early, not the evening before the EvaExam.
- OpenMP: the installed `gcc` supports `-fopenmp`; no module is needed. MPI: load Open MPI with `module load openmpi/4.1.6`, both before compiling and in the job script, and start MPI programs with `srun --mpi=pmi2` (plain `srun` fails in `MPI_Init`).
- *Hint:* What we may call processes, Slurm calls tasks, and MPI calls ranks.

[3] https://slurm.schedmd.com

The running example: computing π

The program `pi.c` approximates $\pi = \int_0^1 \frac{4}{1+x^2}\,dx$ as a sum of `num_steps` rectangles. We parallelize it twice, following the four parallelization steps:

1. **Decomposition:** the sum is split into independent iterations (one rectangle each).
2. **Dependence analysis:** the iterations are independent, except that all results are added into one total `sum`.
3. **Mapping:** the iterations are assigned to threads (Problem 1) or processes (Problem 2).
4. **Parallel programming:** with OpenMP for shared memory (Problem 1) and with MPI for distributed memory (Problem 2). The two differ mainly in how the partial sums are combined.

Problem 1: Shared Memory: π with OpenMP

OpenMP follows the shared-memory programming model with threads. A program starts with one thread. At `#pragma omp parallel`, this thread *forks* a team of threads, and every thread of the team executes the following block. At the end of the block, the threads *join*, and only the original thread continues. All threads share the memory of the process: variables declared *outside* the parallel region are **shared** by all threads, variables declared *inside* are **private** to each thread.

Compile OpenMP programs with `gcc -O3 -fopenmp program.c -o program`. The number of threads is set with the environment variable `OMP_NUM_THREADS`.

1.1 Fork-Join

Compile `regions.c` and run it with 1, 2, and 4 threads (`job_regions.sh`).

1. Which lines are printed once, and which once per thread? How does this match the fork-join model?
2. Run it several times with 4 threads. Is the order of the output lines always the same? Why (not)?

1.2 Parallelizing π with OpenMP

1. Add `#pragma omp parallel for` directly above the `for` loop in `pi.c`. This starts a team of threads and distributes the iterations of the loop among them. Compile it as `pi_omp` and run it with `job_omp.sh`, which runs it with 1, 2, 4, 8, and 20 threads. What do you observe? Why?
2. Change the line to `#pragma omp parallel for reduction(+:sum)`. Now every thread adds into its own private copy of `sum`, and the copies are added together at

the end of the loop. Compile it and submit `job_omp.sh` again. Is the result correct now? Write down the execution times: does the time halve when you double the number of threads? Why (not)?

3. In `pi.c`, `x` is declared before the loop, so it is shared by all threads. Why is this a race condition, even if your runs print the correct value? How do you fix it?

Problem 2: Distributed Memory: π with MPI

MPI follows the distributed-memory / message-passing programming model. An MPI program is started as several *processes* (ranks), possibly on several nodes. All ranks run the same program; `MPI_Comm_rank` tells each rank its number (`0`, `1`, ...), and `MPI_Comm_size` tells it the number of ranks. Every rank has its own private memory: ranks exchange data only by sending and receiving messages.

Compile MPI programs with `mpicc -O3 program.c -o program` (after `module load openmpi/4.1.6`).

2.1 Hello World on Multiple Nodes

Run `hello.c` with 4 ranks on 2 nodes (`job_hello.sh`).

1. Which rank runs on which node? Is the order of the output lines always the same?
2. All ranks run the same program. Why does only one rank print the "MASTER" line?

2.2 Computing π with MPI

`pi_mpi.c` is the MPI version of `pi.c`. Each rank computes a partial sum `local_sum` over its share of the iterations. `MPI_Reduce` then adds up the partial sums of all ranks into `sum` on rank 0.

1. **Mapping:** Which iterations does rank 3 compute when there are 4 ranks?
2. **Beyond one node:** Run `pi_mpi` with 20 ranks on 1 node, 40 ranks on 2 nodes, and 80 ranks on 4 nodes (`job_pi_mpi.sh`). Write down the times and compare them with your OpenMP times with 20 threads. What can MPI do that OpenMP cannot?
3. **Where is the result?** Change `pi_mpi.c` so that *every* rank prints `sum` after `MPI_Reduce`, and run it with 4 ranks: use `job_hello.sh` and replace `./hello` with `./pi_mpi` in its last line. What do the ranks other than rank 0 print? Why?
4. In Problem 1, the threads add into a shared `sum`, and without `reduction` this is a race condition. Can a race condition on `sum` occur in `pi_mpi.c`? Why (not)?

2.3 Message Passing in a Ring: Deadlocks

In `ring.c`, every rank sends an array of `count` integers to its right neighbor and then receives an array from its left neighbor. The message size `count` is passed as a command line argument. Run it with 4 ranks on 2 nodes (`job_ring.sh`). The job has a short time limit, so that a hanging job does not block the nodes for long.

1. Run it with a small message, `./ring 1`. What is printed?
2. Run it with large messages, `./ring 1000000` and `./ring 10000000` (change the number in the last line of `job_ring.sh`). What happens now? Explain the difference. *Hint:* MPI implementations typically send small messages with the *eager* protocol: the message is buffered, and `MPI_Send` returns right away, even if the receiver has not called `MPI_Recv` yet. Large messages are sent with the *rendezvous* protocol: the data is only transferred, and `MPI_Send` only returns, once the receiver has posted the matching receive.
3. Fix the deadlock: let the even ranks first send and then receive, and the odd ranks first receive and then send. Verify that your fix works with `./ring 10000000`.
4. Would an `MPI_Barrier` before `MPI_Send` also fix the deadlock? Why (not)?

Listing 1: `regions.c`: Sequential and parallel regions (Part 1.1).

```c
#include <stdio.h>
#include <omp.h>

int main(void) {
    printf("inside parallel region? %d\n", omp_in_parallel());
    printf("maximum number of threads? %d\n", omp_get_max_threads());

    #pragma omp parallel
    {
        printf("inside parallel region? %d\n", omp_in_parallel());
        printf("my thread id: %d\n", omp_get_thread_num());
    }

    printf("all done\n");
    return 0;
}
```

Listing 2: `job_regions.sh`: Job script for Part 1.1.

```bash
#!/bin/bash
#SBATCH --job-name=regions    # The name of the job
#SBATCH --partition=compute   # The partition of miniHPC
#SBATCH --hint=nomultithread # Disable hardware multithreading
#SBATCH --time=00:02:00       # Requested execution time
#SBATCH --nodes=1             # Requested number of nodes
#SBATCH --ntasks=1            # Requested number of processes
#SBATCH --cpus-per-task=4     # Requested CPUs (threads) per process

for t in 1 2 4; do
    export OMP_NUM_THREADS=$t
    echo "--- $t threads"
    srun ./regions
done
```

Listing 3: `pi.c`: Sequential computation of π (Part 1.2).

```c
#include <stdio.h>
#include <omp.h>

static long num_steps = 10000000000;

int main(void) {
    double x, pi, sum = 0.0;
    double step = 1.0 / (double)num_steps;  // rectangle width
    double start = omp_get_wtime();

    for (long i = 0; i < num_steps; i++) {
        x = (i + 0.5) * step;               // middle of interval i
        sum = sum + 4.0 / (1.0 + x * x);    // F(x_i)
    }
    pi = step * sum;

    double end = omp_get_wtime();
    printf("pi = %.10f, time = %.3f s, threads = %d\n",
           pi, end - start, omp_get_max_threads());
    return 0;
}
```

Listing 4: `job_omp.sh`: Job script for Part 1.2.

```bash
#!/bin/bash
#SBATCH --job-name=pi_omp     # The name of the job
#SBATCH --partition=compute   # The partition of miniHPC
#SBATCH --hint=nomultithread # Disable hardware multithreading
#SBATCH --exclusive           # Whole node for this job only
#SBATCH --time=00:10:00       # Requested execution time
#SBATCH --nodes=1             # Requested number of nodes
#SBATCH --ntasks=1            # Requested number of processes
#SBATCH --cpus-per-task=20    # Requested CPUs (threads) per process

for t in 1 2 4 8 20; do
    export OMP_NUM_THREADS=$t
    srun ./pi_omp
done
```

Listing 5: `hello.c`: MPI Hello World (Part 2.1).

```c
#define MASTER 0
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[]) {
    int numranks, rankid, len;
    char hostname[MPI_MAX_PROCESSOR_NAME];

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numranks);
    MPI_Comm_rank(MPI_COMM_WORLD, &rankid);
    MPI_Get_processor_name(hostname, &len);
    printf("Hello from rank %d on %s\n", rankid, hostname);
    if (rankid == MASTER)
        printf("MASTER: Number of MPI ranks is: %d\n", numranks);
    MPI_Finalize();
    return 0;
}
```

Listing 6: `job_hello.sh`: Job script for Part 2.1.

```
#!/bin/bash
#SBATCH --job-name=hello      # The name of the job
#SBATCH --partition=compute   # The partition of miniHPC
#SBATCH --hint=nomultithread # Disable hardware multithreading
#SBATCH --time=00:02:00       # Requested execution time
#SBATCH --nodes=2             # Requested number of nodes
#SBATCH --ntasks-per-node=2   # Requested processes (ranks) per node
#SBATCH --cpus-per-task=1     # Requested CPUs per process

module load openmpi/4.1.6
srun --mpi=pmi2 ./hello
```

Listing 7: `pi_mpi.c`: Computation of π with MPI (Part 2.2).

```c
#include <stdio.h>
#include <mpi.h>

static long num_steps = 10000000000;

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);   // my rank: 0, ..., size-1
    MPI_Comm_size(MPI_COMM_WORLD, &size);   // number of ranks

    double step = 1.0 / (double)num_steps;  // rectangle width
    double local_sum = 0.0, sum = 0.0;
    double start = MPI_Wtime();

    // Rank r computes iterations r, r + size, r + 2*size, ...
    for (long i = rank; i < num_steps; i += size) {
        double x = (i + 0.5) * step;        // middle of interval i
        local_sum = local_sum + 4.0 / (1.0 + x * x);
    }

    // Add up the local sums of all ranks into sum on rank 0
    MPI_Reduce(&local_sum, &sum, 1, MPI_DOUBLE, MPI_SUM, 0,
               MPI_COMM_WORLD);

    double end = MPI_Wtime();
    if (rank == 0)
        printf("pi = %.10f, time = %.3f s, ranks = %d\n",
               step * sum, end - start, size);
    MPI_Finalize();
    return 0;
}
```

Listing 8: `job_pi_mpi.sh`: Job script for Part 2.2.

```bash
#!/bin/bash
#SBATCH --job-name=pi_mpi     # The name of the job
#SBATCH --partition=compute   # The partition of miniHPC
#SBATCH --hint=nomultithread # Disable hardware multithreading
#SBATCH --exclusive           # Whole nodes for this job only
#SBATCH --time=00:10:00       # Requested execution time
#SBATCH --nodes=4             # Requested number of nodes
#SBATCH --ntasks-per-node=20 # Requested processes (ranks) per node
#SBATCH --cpus-per-task=1     # Requested CPUs per process

module load openmpi/4.1.6
srun --mpi=pmi2 --nodes=1 --ntasks=20 ./pi_mpi   # 1 node
srun --mpi=pmi2 --nodes=2 --ntasks=40 ./pi_mpi   # 2 nodes
srun --mpi=pmi2 --nodes=4 --ntasks=80 ./pi_mpi   # 4 nodes
```

Listing 9: `ring.c`: Message passing in a ring (Part 2.3).

```c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int count = (argc > 1) ? atoi(argv[1]) : 1; // message size
    int *a = malloc(count * sizeof *a);         // send buffer
    int *b = malloc(count * sizeof *b);         // receive buffer
    for (int i = 0; i < count; i++) { a[i] = rank; }

    int right = (rank + 1) % size;
    int left  = (rank - 1 + size) % size;

    MPI_Send(a, count, MPI_INT, right, 1, MPI_COMM_WORLD);
    MPI_Recv(b, count, MPI_INT, left, 1, MPI_COMM_WORLD,
             MPI_STATUS_IGNORE);

    printf("Rank %d received %d ints from rank %d (b[0] = %d)\n",
           rank, count, left, b[0]);

    free(a);
    free(b);
    MPI_Finalize();
    return 0;
}
```

Listing 10: `job_ring.sh`: Job script for Part 2.3.

```
#!/bin/bash
#SBATCH --job-name=ring       # The name of the job
#SBATCH --partition=compute   # The partition of miniHPC
#SBATCH --hint=nomultithread # Disable hardware multithreading
#SBATCH --time=00:02:00       # Requested execution time
#SBATCH --nodes=2             # Requested number of nodes
#SBATCH --ntasks-per-node=2   # Requested processes (ranks) per node
#SBATCH --cpus-per-task=1     # Requested CPUs per process

module load openmpi/4.1.6
srun --mpi=pmi2 ./ring 1      # message size: number of ints
```